

ML-Driven Service Provider Matching Platform

Complete 4-Component Implementation Guide

IEEE Format Research Documentation

Research Team

1970-01-01

This comprehensive document provides complete technical implementation details for a 4-component machine learning platform that matches users with service providers (electricians, plumbers, AC technicians, etc.) in Sri Lanka. The system progressively filters and ranks providers through four intelligent stages: (1) Hybrid Multi-Algorithm Recommendation using Collaborative Filtering, Content-Based Filtering, and BERT semantic matching; (2) Temporal & Contextual Matching incorporating real-time availability, geolocation, and weather; (3) Dynamic Skill Verification & Credibility assessment via background verification and historical performance; and (4) LSTM-Based Sentiment & Review Credibility analysis for aspect-level sentiment extraction and fake review detection. This guide includes mathematical formulations, complete Python implementations, database schemas, API specifications, and evaluation metrics following IEEE research standards.

System Architecture Overview

Platform Objective

The ML-Driven Service Provider Matching Platform addresses the inefficiency of existing Sri Lankan platforms (Hodabass, ikman) which require manual provider selection. This platform automates intelligent provider matching through a 4-stage ML pipeline:

User Request → Component 1 → Component 2 → Component 3 → Component 4 → Top-5 Recommendations

Component Pipeline

1. Component 1: Hybrid Multi-Algorithm Recommendation Engine

- Input: User service request (text)
- Output: Top-20 skill-matched providers
- Algorithms: CF + CBF + BERT (35% + 30% + 35%)
- Time: 2-3 seconds per request

2. Component 2: Temporal & Contextual Matching

- Input: Top-20 from Component 1
- Output: Top-10 available & nearby providers

- Factors: Availability, geolocation, weather, demand
- Time: 1-2 seconds

3. Component 3: Dynamic Skill Verification & Credibility

- Input: Top-10 from Component 2
- Output: Top-8 verified & trustworthy providers
- Verification: Background checks, certifications, historical performance
- Time: 1-2 seconds

4. Component 4: LSTM-Based Sentiment & Review Credibility

- Input: Top-8 from Component 3
- Output: Top-5 final with sentiment insights
- Methods: Bi-LSTM aspect extraction, Isolation Forest fake detection
- Time: 2-3 seconds

Expected Performance Targets

Component	Key Metric	Target
Component 1	Top-5 Accuracy	85%
Component 2	Geolocation Accuracy	90%
Component 3	Credibility Detection	88%
Component 4	Sentiment F1-Score	0.85
End-to-End	User Satisfaction	82%

COMPONENT 1: HYBRID MULTI-ALGORITHM RECOMMENDATION ENGINE

Overview and Objective

Purpose: Generate Top-20 providers based purely on skill match with the user's service request.

Core Question Answered: "Based on this service request, which providers are most likely to successfully solve this problem?"

Output: Ranked list of Top-20 providers with hybrid scores (0-1 scale).

Algorithm 1: Collaborative Filtering (CF)

Theory

Collaborative Filtering assumes that users with similar historical preferences will have similar future preferences. If User A and User B have booked and rated the same providers similarly, they are considered similar. If User B rated Provider X highly but User A hasn't booked with Provider X yet, then Provider X is recommended to User A.

Mathematical Formulation

Given:

- R : User-Provider rating matrix (sparse)
- u_i : Current user
- u_j : Similar user

Step 1: User Similarity (Cosine Similarity)

$$\text{Similarity}(u_i, u_j) = \frac{\sum_p r_{i,p} \cdot r_{j,p}}{\|r_i\| \cdot \|r_j\|}$$

where $r_{i,p}$ is the rating of user i for provider p .

Step 2: Provider Recommendation Score

$$\text{CF_Score}_p = \frac{\sum_{j=1}^k \text{Similarity}(u_i, u_j) \cdot r_{j,p}}{k}$$

where k is the number of similar users (typically 5-10).

Implementation

```
import numpy as np
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler

class CollaborativeFiltering:
    """User-based Collaborative Filtering for provider
    recommendation"""

    def __init__(self, n_neighbors=10):
        self.n_neighbors = n_neighbors
        self.knn_model = None
        self.user_provider_matrix = None
        self.scaler = StandardScaler()

    def fit(self, user_provider_ratings):
```

```

    """
    Fit CF model on user-provider rating matrix

    Args:
        user_provider_ratings: (n_users, n_providers) rating
matrix
    """
    # Normalize ratings
    self.user_provider_matrix =
self.scaler.fit_transform(user_provider_ratings)

    # Build KNN model for user similarity
    self.knn_model = NearestNeighbors(
        n_neighbors=self.n_neighbors,
        metric='cosine'
    )
    self.knn_model.fit(self.user_provider_matrix)

    return self

    def recommend(self, user_id, user_rating_vector,
n_providers_to_rank):
    """
    Generate CF scores for providers for a specific user

    Args:
        user_id: unique identifier
        user_rating_vector: (n_providers,) user's historical
ratings
        n_providers_to_rank: return scores for top N providers

    Returns:
        cf_scores: dict {provider_id: score}
    """
    # Normalize user vector
    user_vector = self.scaler.transform([user_rating_vector])[0]

    # Find k nearest neighbors
    distances, indices = self.knn_model.kneighbors(
        [user_vector], n_neighbors=self.n_neighbors
    )

    # Get neighbor ratings (inverse distance for similarity)
    neighbor_indices = indices[0]
    neighbor_similarities = 1 - distances[0] # Convert distance
to similarity

    # Weighted average of neighbor ratings
    neighbor_ratings = self.user_provider_matrix[neighbor_indices]
    weighted_scores = np.average(

```

```

        neighbor_ratings,
        axis=0,
        weights=neighbor_similarities
    )

    # Normalize to 0-1
    cf_scores = (weighted_scores - weighted_scores.min()) / \
        (weighted_scores.max() - weighted_scores.min() +
1e-8)

    return dict(enumerate(cf_scores))

# Usage Example
cf = CollaborativeFiltering(n_neighbors=10)
user_ratings = np.random.rand(100, 50) # 100 users, 50 providers
cf.fit(user_ratings)
cf_scores = cf.recommend(user_id=0,
    user_rating_vector=user_ratings[0],
    n_providers_to_rank=50)

```

Algorithm 2: Content-Based Filtering (CBF) with TF-IDF

Theory

Content-Based Filtering matches the user's request text with provider profile descriptions using TF-IDF weighting. Important words that distinguish a provider's expertise receive higher weights.

Mathematical Formulation

TF-IDF Score:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

where:

$$\text{TF}(t, d) = \frac{\text{count of term } t \text{ in document } d}{\text{total terms in } d}$$

$$\text{IDF}(t) = \log \left(\frac{\text{total documents}}{1 + \text{documents containing } t} \right)$$

Provider-Request Similarity (Cosine Similarity):

$$\text{CBF_Score}_p = \frac{\text{TF-IDF}_{\text{request}} \cdot \text{TF-IDF}_{\text{provider}}}{\| \text{TF-IDF}_{\text{request}} \| \times \| \text{TF-IDF}_{\text{provider}} \|}$$

Implementation

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

```

```

class ContentBasedFiltering:
    """Content-Based Filtering using TF-IDF for provider matching"""

    def __init__(self, max_features=1000, ngram_range=(1, 2)):
        self.vectorizer = TfidfVectorizer(
            max_features=max_features,
            ngram_range=ngram_range,
            stop_words='english'
        )
        self.provider_tfidf_matrix = None
        self.provider_descriptions = None

    def fit(self, provider_descriptions):
        """
        Fit CBF model on provider descriptions

        Args:
            provider_descriptions: list of provider profile texts
        """
        self.provider_descriptions = provider_descriptions
        self.provider_tfidf_matrix = self.vectorizer.fit_transform(
            provider_descriptions
        )
        return self

    def score_request(self, user_request):
        """
        Score all providers for a user request

        Args:
            user_request: user's service request text

        Returns:
            cbf_scores: dict {provider_id: score}
        """
        # Vectorize request
        request_tfidf = self.vectorizer.transform([user_request])

        # Calculate cosine similarity with all providers
        similarities = cosine_similarity(
            request_tfidf,
            self.provider_tfidf_matrix
        )[0]

        # Normalize to 0-1
        cbf_scores = (similarities - similarities.min()) / \
            (similarities.max() - similarities.min() + 1e-8)

        return dict(enumerate(cbf_scores))

```

```
# Usage Example
cbf = ContentBasedFiltering(max_features=1000)
provider_profiles = [
    "AC repair specialist, compressor diagnostics, 10 years experience",
    "General plumbing services, pipe repair, drainage",
    "Electrical work, wiring, panel installation"
]
cbf.fit(provider_profiles)

user_request = "AC repair, noisy compressor, 5-year-old unit"
cbf_scores = cbf.score_request(user_request)
print(cbf_scores) # {0: 0.95, 1: 0.05, 2: 0.02}
```

Algorithm 3: BERT-Based Deep Semantic Matching

Theory

BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained language model that understands semantic meaning beyond surface-level keywords. It encodes both the user's request and provider profiles into dense vector representations where semantically similar texts have high similarity.

Mathematical Formulation

BERT Encoding:

$$e_{\text{request}} = \text{BERT}(\text{request tokens})$$

$$e_{\text{provider}} = \text{BERT}(\text{provider profile tokens})$$

Semantic Similarity (Cosine):

$$\text{BERT_Score}_p = \cos(e_{\text{request}}, e_{\text{provider}}) = \frac{e_{\text{request}} \cdot e_{\text{provider}}}{\|e_{\text{request}}\| \times \|e_{\text{provider}}\|}$$

Implementation

```
import torch
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

class BERTSemanticMatcher:
    """BERT-based semantic matching for provider recommendations"""

    def __init__(self, model_name='distilbert-base-multilingual-cased'):
        self.device = torch.device('cuda' if torch.cuda.is_available()
                                    else 'cpu')
```

```

        self.model = SentenceTransformer(model_name)
        self.provider_embeddings = None

    def encode_texts(self, texts_list):
        """Encode list of texts to embeddings"""
        embeddings = self.model.encode(
            texts_list,
            convert_to_tensor=True,
            device=self.device
        )
        return embeddings.cpu().numpy()

    def fit(self, provider_descriptions):
        """Fit model by encoding provider descriptions"""
        self.provider_embeddings =
self.encode_texts(provider_descriptions)
        return self

    def score_request(self, user_request):
        """
        Score all providers for a user request

        Args:
            user_request: user's service request text

        Returns:
            bert_scores: dict {provider_id: score}
        """
        # Encode request
        request_embedding = self.encode_texts([user_request])

        # Calculate similarities with all providers
        similarities = cosine_similarity(
            request_embedding,
            self.provider_embeddings
        )[0]

        # Normalize to 0-1
        bert_scores = (similarities - similarities.min()) / \
            (similarities.max() - similarities.min() + 1e-8)

        return dict(enumerate(bert_scores))

# Usage Example
bert_matcher = BERTSemanticMatcher()
provider_profiles = [
    "Expert AC technician handling complex compressor issues",
    "Plumbing specialist for drainage and pipe repairs",
    "Licensed electrician for residential wiring"
]

```



```
bert_matcher.fit(provider_profiles)
```

```
user_request = "My AC is noisy and not cooling properly"  
bert_scores = bert_matcher.score_request(user_request)  
print(bert_scores)  # {0: 0.92, 1: 0.08, 2: 0.05}
```

Hybrid Fusion

Mathematical Formulation

$$\text{Hybrid Score}_p = w_{\text{CF}} \times \text{CF_Score}_p + w_{\text{CBF}} \times \text{CBF_Score}_p + w_{\text{BERT}} \times \text{BERT_Score}_p$$

where $w_{\text{CF}}=0.35$, $w_{\text{CBF}}=0.30$, $w_{\text{BERT}}=0.35$.

Implementation

```
class Component1HybridRecommendation:  
    """Component 1: Hybrid Multi-Algorithm Recommendation Engine"""  
  
    def __init__(self):  
        self.cf = CollaborativeFiltering(n_neighbors=10)  
        self.cbf = ContentBasedFiltering(max_features=1000)  
        self.bert = BERTSemanticMatcher()  
  
        # Weights  
        self.weights = {  
            'cf': 0.35,  
            'cbf': 0.30,  
            'bert': 0.35  
        }  
  
    def fit(self, user_ratings_matrix, provider_descriptions):  
        """Train all three algorithms"""  
        self.cf.fit(user_ratings_matrix)  
        self.cbf.fit(provider_descriptions)  
        self.bert.fit(provider_descriptions)  
        return self  
  
    def recommend_top_k(self, user_id, user_rating_vector,  
                        user_request, k=20):  
        """  
        Generate Top-K provider recommendations  
  
        Args:  
            user_id: unique user identifier  
            user_rating_vector: user's historical ratings  
            user_request: user's current service request (text)  
            k: number of top providers to return  
  
        Returns:  
            top_k_providers: list of (provider_id, hybrid_score)
```

```

    """
    # Get scores from each algorithm
    cf_scores = self.cf.recommend(user_id, user_rating_vector,
                                  len(user_rating_vector))
    cbf_scores = self.cbf.score_request(user_request)
    bert_scores = self.bert.score_request(user_request)

    # Fuse scores
    n_providers = len(user_rating_vector)
    hybrid_scores = {}

    for p in range(n_providers):
        hybrid_score = (
            self.weights['cf'] * cf_scores.get(p, 0) +
            self.weights['cbf'] * cbf_scores.get(p, 0) +
            self.weights['bert'] * bert_scores.get(p, 0)
        )
        hybrid_scores[p] = hybrid_score

    # Rank and return top-k
    sorted_providers = sorted(
        hybrid_scores.items(),
        key=lambda x: x[1],
        reverse=True
    )

    top_k = sorted_providers[:k]

    return top_k

# Usage Example
comp1 = Component1HybridRecommendation()

# Training
user_ratings = np.random.rand(100, 50)
provider_profiles = [
    f"Provider {i}: AC repair, electrical, plumbing specialties"
    for i in range(50)
]
comp1.fit(user_ratings, provider_profiles)

# Inference
user_request = "AC repair, noisy compressor, Colombo 7"
top_20 = comp1.recommend_top_k(
    user_id=0,
    user_rating_vector=user_ratings[0],
    user_request=user_request,
    k=20
)

```

```
print("Component 1 Output (Top-20):")
for rank, (provider_id, score) in enumerate(top_20, 1):
    print(f" {rank}. Provider {provider_id}: {score:.4f}")
```

Component 1 Performance Metrics

Metric	Target	Definition
Accuracy@5	85%	Top-5 contains user's eventual choice
Precision@20	90%	90% of Top-20 have good ratings
NDCG@5	0.85	Ranking quality (Normalized Discounted Cumulative Gain)
Coverage	60%	Fraction of providers appearing in recommendations
Diversity	High	Variance in experience, price, location

COMPONENT 2: TEMPORAL & CONTEXTUAL MATCHING

Overview and Objective

Purpose: Refine Top-20 from Component 1 by considering real-time factors: provider availability, geolocation distance, weather impact, and demand dynamics.

Core Question Answered: “Of the skill-matched providers, which are currently available, nearby, and not overwhelmed with demand?”

Input: Top-20 from Component 1. **Output:** Top-10 providers available and contextually suitable.

Module 1: Availability Matching

Theory

Provider availability depends on:

- Current online status
- Calendar availability (booked slots)
- Service response time SLA (Service Level Agreement)
- Historical average response time

Mathematical Formulation

$$\text{Availability_Score}_p = \alpha \cdot \text{Online}(p) + \beta \cdot \frac{\text{Free_Slots}(p)}{\text{Max_Daily_Slots}} + \gamma \cdot e^{-\lambda \cdot \text{Avg_Response_Time}(p)}$$

where $\alpha=0.3$, $\beta=0.4$, $\gamma=0.3$, and $\lambda=0.5$ (decay rate).

Implementation

```
from datetime import datetime, timedelta
import numpy as np

class AvailabilityMatcher:
    """Match providers based on current availability"""

    def __init__(self):
        self.provider_availability = {}

    def update_provider_status(self, provider_id, status_data):
        """
        Update provider availability status

        Args:
            provider_id: unique identifier
            status_data: {
                'is_online': bool,
                'booked_slots': list of datetime slots,
                'max_daily_slots': int,
                'avg_response_time_minutes': float
            }
        """
        self.provider_availability[provider_id] = status_data

    def calculate_availability_score(self, provider_id, current_time):
        """
        Calculate availability score (0-1)

        Args:
            provider_id: unique identifier
            current_time: datetime object

        Returns:
            availability_score: float (0-1)
        """
        if provider_id not in self.provider_availability:
            return 0.0

        data = self.provider_availability[provider_id]

        # Component 1: Online status (0 or 1)
        online_score = 1.0 if data['is_online'] else 0.0
```

```

# Component 2: Available slots ratio
booked_count = len(data['booked_slots'])
max_slots = data['max_daily_slots']
available_slots = max(0, max_slots - booked_count)
slots_score = available_slots / (max_slots + 1e-8)

# Component 3: Response time decay
avg_response = data['avg_response_time_minutes']
response_score = np.exp(-0.5 * avg_response / 60) # Decay per
hour

# Weighted combination
weights = {'online': 0.3, 'slots': 0.4, 'response': 0.3}
availability_score = (
    weights['online'] * online_score +
    weights['slots'] * slots_score +
    weights['response'] * response_score
)

return min(1.0, max(0.0, availability_score))

# Usage Example
availability = AvailabilityMatcher()
availability.update_provider_status('P001', {
    'is_online': True,
    'booked_slots': [datetime.now() + timedelta(hours=1),
                     datetime.now() + timedelta(hours=2)],
    'max_daily_slots': 8,
    'avg_response_time_minutes': 15
})
score = availability.calculate_availability_score('P001',
datetime.now())
print(f"Availability Score: {score:.3f}")

```

Module 2: Geolocation Matching

Theory

Distance affects service quality: closer providers arrive faster, reducing wait time and increasing user satisfaction. We use Haversine distance formula for great-circle distance.

Mathematical Formulation

Haversine Distance:

$$d = 2R \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta \phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta \lambda}{2} \right)} \right)$$

where $R=6371$ km (Earth's radius), ϕ is latitude, λ is longitude.

Distance-Based Score (Inverse Decay):

$$\text{Geo_Score}_p = e^{-\mu \cdot d_p}$$

where d_p is distance in km and μ is decay rate (typically 0.2-0.5).

Implementation

```
from math import radians, sin, cos, sqrt, atan2
import numpy as np

class GeolocationMatcher:
    """Match providers based on geolocation proximity"""

    EARTH_RADIUS_KM = 6371

    @staticmethod
    def haversine_distance(lat1, lon1, lat2, lon2):
        """
        Calculate great-circle distance between two points

        Args:
            lat1, lon1: user location (decimal degrees)
            lat2, lon2: provider location (decimal degrees)

        Returns:
            distance: in kilometers
        """
        lat1, lon1, lat2, lon2 = map(radians, [lat1, lon1, lat2,
lon2])

        dlat = lat2 - lat1
        dlon = lon2 - lon1

        a = sin(dlat/2)**2 + cos(lat1) * cos(lat2) * sin(dlon/2)**2
        c = 2 * atan2(sqrt(a), sqrt(1-a))
        distance = GeolocationMatcher.EARTH_RADIUS_KM * c

        return distance

    def __init__(self, decay_rate=0.3):
        self.decay_rate = decay_rate # Higher = faster decay
        self.provider_locations = {}

    def add_provider_location(self, provider_id, latitude, longitude):
        """Store provider geolocation"""
        self.provider_locations[provider_id] = (latitude, longitude)

    def calculate_geo_score(self, user_lat, user_lon, provider_id):
        """
        Calculate geolocation score (0-1)

```

```

    Args:
        user_lat, user_lon: user location
        provider_id: provider identifier

    Returns:
        geo_score: float (0-1)
    """
    if provider_id not in self.provider_locations:
        return 0.0

    provider_lat, provider_lon =
self.provider_locations[provider_id]

    # Calculate distance
    distance = self.haversine_distance(
        user_lat, user_lon,
        provider_lat, provider_lon
    )

    # Convert to score (exponential decay)
    # At distance=0: score=1.0
    # At distance=5km: score~0.22
    geo_score = np.exp(-self.decay_rate * distance)

    return geo_score

# Usage Example
geo_matcher = GeolocationMatcher(decay_rate=0.3)

# Colombo 7 providers
geo_matcher.add_provider_location('P001', 6.9271, 79.8612)
geo_matcher.add_provider_location('P002', 6.9300, 79.8650)
geo_matcher.add_provider_location('P003', 6.9150, 79.8700)

# User in Colombo 7
user_lat, user_lon = 6.9270, 79.8610

scores = {
    'P001': geo_matcher.calculate_geo_score(user_lat, user_lon,
    'P001'),
    'P002': geo_matcher.calculate_geo_score(user_lat, user_lon,
    'P002'),
    'P003': geo_matcher.calculate_geo_score(user_lat, user_lon,
    'P003')
}
print(f"Geolocation Scores: {scores}")

```

Module 3: Weather Impact Assessment

Theory

Weather affects service urgency and availability. Extreme weather (heavy rain, heat) increases demand for certain services (water leaks, AC repair) while potentially reducing provider availability.

Mathematical Formulation

$$\text{Weather_Impact}_p = w_{\text{demand}} \times \frac{\text{Demand_Multiplier}_{\text{weather}}}{\text{Demand}_{\text{current}}(p)} + w_{\text{avail}} \times \text{Availability_Reduction}_{\text{weather}}(p)$$

Implementation

```
import json
from datetime import datetime

class WeatherMatcher:
    """Assess weather impact on service demand and provider
    availability"""

    def __init__(self):
        self.demand_multipliers = {
            'ac_repair': {
                'hot': 3.0,          # Extreme heat increases AC repair
demand 3x
                'rainy': 0.5,
                'normal': 1.0
            },
            'plumbing': {
                'hot': 0.5,
                'rainy': 3.0,      # Heavy rain increases plumbing
demand 3x
                'normal': 1.0
            },
            'electrical': {
                'hot': 1.2,
                'rainy': 2.0,      # Rain increases electrical faults
                'normal': 1.0
            }
        }

        self.availability_reduction = {
            'hot': 0.9,          # 10% less availability in heat
            'rainy': 0.8,       # 20% less availability in rain
            'normal': 1.0
        }

    def get_current_weather(self, location_lat, location_lon):
        """
```



```

Fetch current weather
In production, integrate OpenWeatherMap API
"""
# Simulated
return {
    'condition': 'hot', # 'hot', 'rainy', 'normal'
    'temperature': 35,
    'humidity': 70,
    'rainfall': 0
}

def calculate_weather_impact(self, service_type, location_lat,
                             location_lon, current_availability):
    """
    Calculate weather impact on service

    Returns:
        impact_score: multiplier for availability (0.5 to 3.0)
    """
    weather = self.get_current_weather(location_lat, location_lon)
    condition = weather['condition']

    # Get demand multiplier
    demand_mult = self.demand_multipliers.get(service_type,
{ }).get(
        condition, 1.0
    )

    # Get availability reduction
    avail_reduction = self.availability_reduction.get(condition,
1.0)

    # Combined impact
    impact = demand_mult * avail_reduction

    return impact

# Usage Example
weather_matcher = WeatherMatcher()
impact = weather_matcher.calculate_weather_impact(
    service_type='ac_repair',
    location_lat=6.9270,
    location_lon=79.8610,
    current_availability=0.8
)
print(f"Weather Impact Multiplier: {impact:.2f}")

```

Module 4: Demand Dynamics and Load Balancing

Theory

Providers with very high demand may have longer wait times, reducing service quality. Load balancing considers current provider load and suggests less-loaded alternatives.

Implementation

```
class DemandDynamicsMatcher:
    """Handle demand dynamics and load balancing"""

    def __init__(self):
        self.provider_load = {}
        self.max_daily_jobs = 8

    def update_provider_load(self, provider_id, current_jobs):
        """Update current job count for provider"""
        self.provider_load[provider_id] = current_jobs

    def calculate_load_score(self, provider_id):
        """
        Calculate load score (0-1)
        Lower load = higher score
        """
        if provider_id not in self.provider_load:
            return 0.5

        current_jobs = self.provider_load[provider_id]
        load_ratio = current_jobs / self.max_daily_jobs

        # Sigmoid curve: high load reduces score
        score = 1.0 / (1.0 + np.exp(5 * (load_ratio - 0.5)))
        return score

# Usage Example
demand = DemandDynamicsMatcher()
demand.update_provider_load('P001', 5) # 5 of 8 slots filled
demand.update_provider_load('P002', 2)
score_p1 = demand.calculate_load_score('P001')
score_p2 = demand.calculate_load_score('P002')
print(f"P001 Load Score: {score_p1:.3f}, P002 Load Score: {score_p2:.3f}")
```

Component 2 Integration

```
class Component2TemporalContextual:
    """Component 2: Temporal & Contextual Matching"""

    def __init__(self):
        self.availability = AvailabilityMatcher()
        self.geolocation = GeolocationMatcher(decay_rate=0.3)
```

```

self.weather = WeatherMatcher()
self.demand = DemandDynamicsMatcher()

self.weights = {
    'availability': 0.30,
    'geolocation': 0.35,
    'weather': 0.15,
    'demand': 0.20
}

def refine_top_20(self, top_20_providers, user_lat, user_lon,
                 service_type, k=10):
    """
    Refine Top-20 to Top-10 using temporal and contextual factors

    Args:
        top_20_providers: list from Component 1
        user_lat, user_lon: user geolocation
        service_type: type of service
        k: return top-k

    Returns:
        top_10_providers: refined list
    """
    refined_scores = []

    for provider_id, compl_score in top_20_providers:
        # Calculate contextual scores
        avail_score =
self.availability.calculate_availability_score(
    provider_id, datetime.now()
)
        geo_score = self.geolocation.calculate_geo_score(
    user_lat, user_lon, provider_id
)
        weather_impact = self.weather.calculate_weather_impact(
    service_type, user_lat, user_lon, avail_score
)
        load_score = self.demand.calculate_load_score(provider_id)

        # Weighted combination
        contextual_score = (
            self.weights['availability'] * avail_score +
            self.weights['geolocation'] * geo_score +
            self.weights['weather'] * (1.0 / weather_impact) +
            self.weights['demand'] * load_score
        )

        # Final score = Component 1 + Component 2
        final_score = 0.7 * compl_score + 0.3 * contextual_score

```

```

        refined_scores.append((provider_id, final_score))

    # Rank and return top-k
    ranked = sorted(refined_scores, key=lambda x: x[1],
reverse=True)
    top_k = ranked[:k]

    return top_k

# Usage Example
comp2 = Component2TemporalContextual()
top_20 = [('P001', 0.89), ('P002', 0.87), ('P003', 0.85)]
top_10 = comp2.refine_top_20(
    top_20,
    user_lat=6.9270,
    user_lon=79.8610,
    service_type='ac_repair',
    k=10
)
print("Component 2 Output (Top-10):")
for rank, (pid, score) in enumerate(top_10, 1):
    print(f" {rank}. {pid}: {score:.4f}")

```

COMPONENT 3: DYNAMIC SKILL VERIFICATION & CREDIBILITY

Overview and Objective

Purpose: Verify provider credentials and assess trustworthiness based on background checks, certifications, historical performance, and user feedback.

Core Question Answered: “Is this provider trustworthy and genuinely qualified?”

Input: Top-10 from Component 2. **Output:** Top-8 verified and credible providers.

Module 1: Credential Verification

Implementation

```

from datetime import datetime, timedelta
from enum import Enum

```

```

class CertificationStatus(Enum):
    VERIFIED = 1
    PENDING = 0.5
    EXPIRED = 0
    NONE = 0

```

```

class CredentialVerifier:
    """Verify provider credentials and certifications"""

```

```

def __init__(self):
    self.provider_credentials = {}

def add_provider_credentials(self, provider_id, credentials):
    """
    Add provider credentials

    Args:
        credentials: {
            'certifications': [{'name': str, 'issued_date': date,
                                'expiry_date': date, 'verified':
bool}, ...],
            'years_experience': int,
            'background_check_status': str, # VERIFIED, PENDING,
FAILED
            'insurance_status': str # ACTIVE, INACTIVE
        }
    """
    self.provider_credentials[provider_id] = credentials

def verify_certification(self, provider_id, certification_name):
    """Check if provider has active certification"""
    if provider_id not in self.provider_credentials:
        return CertificationStatus.NONE

    creds = self.provider_credentials[provider_id]
    certs = creds.get('certifications', [])

    for cert in certs:
        if cert['name'].lower() == certification_name.lower():
            if not cert['verified']:
                return CertificationStatus.PENDING

            expiry = cert['expiry_date']
            if expiry and expiry < datetime.now().date():
                return CertificationStatus.EXPIRED

            return CertificationStatus.VERIFIED

    return CertificationStatus.NONE

def calculate_credential_score(self, provider_id):
    """
    Calculate credential score (0-1)

    Returns:
        score: float (0-1)
    """
    if provider_id not in self.provider_credentials:

```

```

        return 0.0

    creds = self.provider_credentials[provider_id]

    # Component 1: Certifications
    cert_statuses = [
        cert.get('verified', False)
        for cert in creds.get('certifications', [])
    ]
    cert_score = sum(cert_statuses) / (len(cert_statuses) + 1e-8)
    if cert_statuses else 0

    # Component 2: Experience
    years = creds.get('years_experience', 0)
    exp_score = min(1.0, years / 10.0) # 10+ years = max score

    # Component 3: Background check
    bg_status = creds.get('background_check_status', 'NONE')
    bg_score = 1.0 if bg_status == 'VERIFIED' else 0.0

    # Component 4: Insurance
    insurance = creds.get('insurance_status', 'INACTIVE')
    insurance_score = 1.0 if insurance == 'ACTIVE' else 0.5

    # Weighted combination
    weights = {
        'certifications': 0.25,
        'experience': 0.25,
        'background': 0.30,
        'insurance': 0.20
    }

    credential_score = (
        weights['certifications'] * cert_score +
        weights['experience'] * exp_score +
        weights['background'] * bg_score +
        weights['insurance'] * insurance_score
    )

    return credential_score

# Usage Example
verifier = CredentialVerifier()
verifier.add_provider_credentials('P001', {
    'certifications': [
        {'name': 'AC Technician', 'issued_date': datetime(2020, 1,
1).date(),
        'expiry_date': datetime(2026, 1, 1).date(), 'verified': True}
    ],
    'years_experience': 8,

```

```

        'background_check_status': 'VERIFIED',
        'insurance_status': 'ACTIVE'
    })

score = verifier.calculate_credential_score('P001')
print(f"Credential Score: {score:.3f}")

```

Module 2: Historical Performance Analysis

Theory

Provider's past performance is predictive of future service quality. Key metrics:

- Completion rate: percentage of jobs completed on schedule
- Customer rating distribution
- Dispute resolution rate
- Service consistency (variance in ratings)

Implementation

```

import numpy as np
from scipy import stats

class PerformanceAnalyzer:
    """Analyze provider historical performance"""

    def __init__(self):
        self.provider_history = {}

    def add_job_history(self, provider_id, jobs_list):
        """
        Add provider job history

        Args:
            jobs_list: list of {
                'job_id': str,
                'rating': float (1-5),
                'completion_time_hours': float,
                'scheduled_time_hours': float,
                'customer_comments': str,
                'dispute_flag': bool
            }
        """
        self.provider_history[provider_id] = jobs_list

    def calculate_performance_score(self, provider_id):
        """Calculate overall performance score (0-1)"""
        if provider_id not in self.provider_history:
            return 0.5 # Default for new providers

```

```

jobs = self.provider_history[provider_id]

if len(jobs) == 0:
    return 0.5

# Metric 1: Average rating
ratings = [j['rating'] for j in jobs]
avg_rating = np.mean(ratings) / 5.0 # Normalize to 0-1

# Metric 2: Completion rate (on-time jobs)
on_time_jobs = sum(
    1 for j in jobs
    if j['completion_time_hours'] <= j['scheduled_time_hours']
)
completion_rate = on_time_jobs / len(jobs)

# Metric 3: Rating consistency (low variance = consistent)
rating_std = np.std(ratings)
consistency = 1.0 / (1.0 + rating_std) # Sigmoid: high
consistency = high score

# Metric 4: Dispute rate
dispute_rate = sum(1 for j in jobs if j['dispute_flag']) /
len(jobs)
dispute_score = 1.0 - dispute_rate

# Weighted combination
weights = {
    'rating': 0.35,
    'completion': 0.25,
    'consistency': 0.20,
    'disputes': 0.20
}

performance_score = (
    weights['rating'] * avg_rating +
    weights['completion'] * completion_rate +
    weights['consistency'] * consistency +
    weights['disputes'] * dispute_score
)

return min(1.0, max(0.0, performance_score))

# Usage Example
analyzer = PerformanceAnalyzer()
jobs = [
    {'job_id': 'J1', 'rating': 5, 'completion_time_hours': 1,
    'scheduled_time_hours': 2, 'customer_comments': 'Great',
    'dispute_flag': False},

```



```

        {'job_id': 'J2', 'rating': 4, 'completion_time_hours': 1.5,
'scheduled_time_hours': 2, 'customer_comments': 'Good',
'dispute_flag': False},
        {'job_id': 'J3', 'rating': 5, 'completion_time_hours': 2,
'scheduled_time_hours': 2, 'customer_comments': 'Excellent',
'dispute_flag': False}
    ]
    analyzer.add_job_history('P001', jobs)
    score = analyzer.calculate_performance_score('P001')
    print(f"Performance Score: {score:.3f}")

```

Module 3: Fraud Detection

Implementation

```

from sklearn.ensemble import IsolationForest

class FraudDetector:
    """Detect fraudulent provider behavior"""

    def __init__(self, contamination=0.05):
        self.model = IsolationForest(contamination=contamination,
random_state=42)
        self.is_fitted = False

    def extract_fraud_features(self, provider_data):
        """
        Extract features for fraud detection

        Args:
            provider_data: {
                'jobs_count': int,
                'avg_rating': float,
                'rating_variance': float,
                'completion_rate': float,
                'dispute_count': int,
                'refund_count': int,
                'customer_comments_length': float (avg),
                'service_type_changes': int
            }
        """
        features = [
            provider_data['jobs_count'],
            provider_data['avg_rating'],
            provider_data['rating_variance'],
            provider_data['completion_rate'],
            provider_data['dispute_count'],
            provider_data['refund_count'],
            provider_data['customer_comments_length'],
            provider_data['service_type_changes']
        ]

```

```

        return np.array(features).reshape(1, -1)

    def fit(self, providers_list):
        """Train fraud detector"""
        features_list = []
        for prov in providers_list:
            features = self.extract_fraud_features(prov)
            features_list.append(features[0])

        features_array = np.array(features_list)
        self.model.fit(features_array)
        self.is_fitted = True
        return self

    def is_fraudulent(self, provider_data):
        """
        Detect if provider is fraudulent

        Returns:
            is_fraud: bool
            fraud_score: float (-1 to 1, lower = more anomalous)
        """
        features = self.extract_fraud_features(provider_data)
        prediction = self.model.predict(features)[0]
        score = self.model.score_samples(features)[0]

        is_fraud = prediction == -1

        return is_fraud, score

# Usage Example
detector = FraudDetector(contamination=0.05)
providers = [
    {'jobs_count': 100, 'avg_rating': 4.8, 'rating_variance': 0.2,
     'completion_rate': 0.98, 'dispute_count': 0, 'refund_count': 0,
     'customer_comments_length': 50, 'service_type_changes': 0},
    {'jobs_count': 50, 'avg_rating': 5.0, 'rating_variance': 0.0, #
     'completion_rate': 1.0, 'dispute_count': 0, 'refund_count': 0,
     'customer_comments_length': 10, 'service_type_changes': 0}
]
detector.fit(providers)
is_fraud, score = detector.is_fraudulent(providers[1])
print(f"Fraudulent: {is_fraud}, Score: {score:.3f}")

```

Component 3 Integration

```

class Component3DynamicCredibility:
    """Component 3: Dynamic Skill Verification & Credibility"""

    def __init__(self):

```

```

self.verifier = CredentialVerifier()
self.analyzer = PerformanceAnalyzer()
self.fraud_detector = FraudDetector()

self.weights = {
    'credentials': 0.30,
    'performance': 0.40,
    'fraud': 0.30
}

def assess_credibility(self, provider_id, provider_data, top_10):
    """
    Assess provider credibility

    Returns:
        credibility_score: float (0-1)
    """
    # Component 1: Credentials
    cred_score =
self.verifier.calculate_credential_score(provider_id)

    # Component 2: Performance
    perf_score =
self.analyzer.calculate_performance_score(provider_id)

    # Component 3: Fraud detection
    is_fraud, fraud_score =
self.fraud_detector.is_fraudulent(provider_data)
    fraud_penalty = 0.0 if not is_fraud else 0.3

    # Weighted combination
    credibility_score = (
        self.weights['credentials'] * cred_score +
        self.weights['performance'] * perf_score +
        self.weights['fraud'] * (1.0 - fraud_penalty)
    )

    return min(1.0, max(0.0, credibility_score))

def refine_top_10_to_top_8(self, top_10, provider_database):
    """
    Refine Top-10 to Top-8 based on credibility

    Returns:
        top_8: list of (provider_id, credibility_score)
    """
    credibility_scores = []

    for provider_id, comp2_score in top_10:
        provider_data = provider_database.get(provider_id, {})

```

```

        credibility_score = self.assess_credibility(
            provider_id, provider_data, top_10
        )

        # Final score = 60% Component 2 + 40% Credibility
        final_score = 0.6 * comp2_score + 0.4 * credibility_score

        credibility_scores.append((provider_id, final_score))

    # Rank and return top-8
    ranked = sorted(credibility_scores, key=lambda x: x[1],
reverse=True)
    top_8 = ranked[:8]

    return top_8

# Usage Example
comp3 = Component3DynamicCredibility()
top_10 = [('P001', 0.88), ('P002', 0.85)]
provider_db = {
    'P001': {'jobs_count': 100, 'avg_rating': 4.8, ...},
    'P002': {'jobs_count': 50, 'avg_rating': 4.5, ...}
}
top_8 = comp3.refine_top_10_to_top_8(top_10, provider_db)
print("Component 3 Output (Top-8):")
for rank, (pid, score) in enumerate(top_8, 1):
    print(f" {rank}. {pid}: {score:.4f}")

```

COMPONENT 4: LSTM-BASED SENTIMENT & REVIEW CREDIBILITY

Overview and Objective

Purpose: Extract aspect-level sentiment from reviews and detect fake reviews, producing final Top-5 recommendations with sentiment-enriched profiles.

Core Question Answered: “What is the genuine customer sentiment about this provider across multiple dimensions?”

Input: Top-8 from Component 3. **Output:** Top-5 final recommendations with detailed sentiment profiles.

Module 1: Review Preprocessing

```

import re
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

class ReviewPreprocessor:

```

```

"""Preprocess customer reviews for NLP analysis"""

def __init__(self, lang='english'):
    self.stop_words = set(stopwords.words(lang))
    self.aspect_keywords = {
        'punctuality': ['on time', 'late', 'delay', 'quick',
'slow', 'fast', 'rush'],
        'quality': ['good quality', 'poor work', 'excellent',
'bad', 'perfect', 'broken', 'shoddy'],
        'communication': ['friendly', 'rude', 'helpful',
'unclear', 'responsive', 'silent', 'polite'],
        'professionalism': ['professional', 'unprofessional',
'neat', 'messy', 'attentive', 'careless'],
        'price': ['expensive', 'cheap', 'fair price',
'overpriced', 'value for money', 'costly']
    }

    def clean_text(self, text):
        """Clean and normalize text"""
        text = text.lower()
        text = re.sub(r'^a-z\s', '', text)
        text = re.sub(r'\s+', ' ', text)
        return text.strip()

    def tokenize_review(self, text):
        """Tokenize and remove stopwords"""
        cleaned = self.clean_text(text)
        tokens = word_tokenize(cleaned)
        tokens = [t for t in tokens if t not in self.stop_words]
        return tokens

    def preprocess_batch(self, reviews_list):
        """Preprocess batch of reviews"""
        processed = []
        for review in reviews_list:
            tokens = self.tokenize_review(review)
            processed.append({
                'original': review,
                'tokens': tokens,
                'length': len(tokens),
                'cleaned': self.clean_text(review)
            })
        return processed

# Usage Example
preprocessor = ReviewPreprocessor()
reviews = [
    "Provider arrived on time and fixed AC perfectly",
    "Late arrival but excellent work quality"
]

```

```
processed = preprocessor.preprocess_batch(reviews)
print(processed)
```

Module 2: Bi-LSTM Aspect Sentiment Extraction

```
import torch
import torch.nn as nn
from torch.nn.utils.rnn import pad_sequence

class AspectSentimentLSTM(nn.Module):
    """Bi-LSTM for aspect-level sentiment extraction"""

    def __init__(self, vocab_size=5000, embedding_dim=100,
hidden_dim=128):
        super(AspectSentimentLSTM, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim,
padding_idx=0)
        self.lstm = nn.LSTM(
            embedding_dim, hidden_dim, batch_first=True,
            bidirectional=True, num_layers=2, dropout=0.3
        )
        self.attention = nn.MultiheadAttention(hidden_dim*2,
num_heads=4)

        # Aspect classifiers
        self.aspect_heads = nn.ModuleDict({
            'punctuality': nn.Linear(hidden_dim*2, 3),
            'quality': nn.Linear(hidden_dim*2, 3),
            'communication': nn.Linear(hidden_dim*2, 3),
            'professionalism': nn.Linear(hidden_dim*2, 3),
            'price': nn.Linear(hidden_dim*2, 3)
        })
        self.aspects = list(self.aspect_heads.keys())

    def forward(self, token_ids):
        """Forward pass"""
        embedded = self.embedding(token_ids)
        lstm_out, _ = self.lstm(embedded)
        attn_out, _ = self.attention(lstm_out, lstm_out, lstm_out)
        features = attn_out.mean(dim=1)

        aspect_sentiments = {}
        for aspect in self.aspects:
            logits = self.aspect_heads[aspect](features)
            probs = torch.softmax(logits, dim=1)
            # Score: -1 (negative), 0 (neutral), +1 (positive)
            sentiment = (probs[:, 2] - probs[:, 0])
            aspect_sentiments[aspect] = (sentiment.item() + 1) / 2 #
        # Normalize to 0-1

        return aspect_sentiments
```

```

class AspectSentimentExtractor:
    """Wrapper for sentiment extraction"""

    def __init__(self):
        self.device = torch.device('cuda' if torch.cuda.is_available()
else 'cpu')
        self.model = AspectSentimentLSTM()
        self.model.eval()

    def extract_sentiments(self, reviews_tokens):
        """Extract aspect sentiments from reviews"""
        sentiments_dict = {}

        with torch.no_grad():
            for idx, tokens in enumerate(reviews_tokens):
                token_tensor =
torch.LongTensor([tokens]).to(self.device)
                aspect_sentiments = self.model(token_tensor)
                sentiments_dict[idx] = aspect_sentiments

        return sentiments_dict

# Usage Example
extractor = AspectSentimentExtractor()
sample_tokens = [[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]]
sentiments = extractor.extract_sentiments(sample_tokens)
print(sentiments)

```

Module 3: Fake Review Detection (Isolation Forest)

```

from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

class FakeReviewDetector:
    """Detect fake or suspicious reviews"""

    def __init__(self, contamination=0.1):
        self.model = IsolationForest(contamination=contamination,
random_state=42)
        self.scaler = StandardScaler()

    def extract_review_features(self, review_data):
        """Extract anomaly detection features"""
        text = review_data.get('text', '')
        rating = review_data.get('rating', 3)

        features = [
            len(text.split()),           # Length
            text.count('!'),             # Exclamations
            sum(1 for w in text.split() if w.isupper()), # CAPS words

```

```

        sum(1 for i in range(len(text)-2)
            if i+2 < len(text) and text[i] == text[i+1] ==
text[i+2]), # Repeated chars
        1 if rating in [1, 5] else 0, # Extreme
rating
        len(set(text.lower().split())) / (len(text.split()) + 1e-
8), # Diversity
        text.count('http') + text.count('www') # URLs
    ]
    return np.array(features).reshape(1, -1)

def fit_detector(self, reviews_list):
    """Train detector"""
    all_features = []
    for review in reviews_list:
        features = self.extract_review_features(review)
        all_features.append(features[0])

    all_features = np.array(all_features)
    all_features_scaled = self.scaler.fit_transform(all_features)
    self.model.fit(all_features_scaled)
    return self

def detect_fake_reviews(self, reviews_list):
    """Detect fake reviews"""
    results = []

    for idx, review in enumerate(reviews_list):
        features = self.extract_review_features(review)
        features_scaled = self.scaler.transform(features)

        prediction = self.model.predict(features_scaled)[0]
        anomaly_score = self.model.score_samples(features_scaled)

[0]
        credibility = 1 / (1 + np.exp(-anomaly_score))

        results.append({
            'review_idx': idx,
            'is_fake': prediction == -1,
            'credibility_score': float(credibility)
        })

    return results

# Usage Example
detector = FakeReviewDetector(contamination=0.1)
reviews = [
    {'text': 'Excellent service, on time, great quality', 'rating':
5},
    {'text': 'S0000000 GOOD!!!!!!', 'rating': 5}

```



```

]
detector.fit_detector(reviews)
fake_results = detector.detect_fake_reviews(reviews)
print(fake_results)

```

Module 4: Sentiment Aggregation

```

class SentimentAggregator:
    """Aggregate sentiments and adjust final ranking"""

    def __init__(self):
        self.aspect_weights = {
            'quality': 0.30,
            'punctuality': 0.25,
            'communication': 0.15,
            'professionalism': 0.15,
            'price': 0.15
        }

    def aggregate_provider_sentiments(self, aspect_sentiments_list,
                                     credibility_scores):
        """Aggregate multiple review sentiments"""
        # Weight by credibility
        credible_reviews = [
            (sent, cred) for sent, cred in zip(aspect_sentiments_list,
                                               credibility_scores)
            if cred > 0.5
        ]

        if not credible_reviews:
            return {aspect: 0.5 for aspect in self.aspect_weights}

        aggregated = {}
        for aspect in self.aspect_weights.keys():
            weighted_sum = sum(
                review[aspect] * cred
                for review, cred in credible_reviews
            )
            total_cred = sum(cred for _, cred in credible_reviews)
            aggregated[aspect] = weighted_sum / total_cred

        # Overall sentiment
        overall = sum(
            aggregated[a] * w for a, w in self.aspect_weights.items()
        )
        aggregated['overall'] = overall
        aggregated['num_credible_reviews'] = len(credible_reviews)

        return aggregated

```

```

def adjust_final_ranking(self, comp3_score, sentiment_scores,
                        sentiment_weight=0.20):
    """Adjust final score with sentiment"""
    sentiment_boost = sentiment_scores.get('overall', 0.5)
    final_score = (
        (1 - sentiment_weight) * comp3_score +
        sentiment_weight * sentiment_boost
    )
    return final_score

def generate_explanation(self, provider_name, sentiment_scores):
    """Generate readable sentiment explanation"""
    explanations = []
    for aspect in self.aspect_weights.keys():
        score = sentiment_scores.get(aspect, 0.5)
        if score > 0.7:
            text = f"Excellent {aspect}"
        elif score > 0.5:
            text = f"Good {aspect}"
        else:
            text = f"Fair {aspect}"
        explanations.append(text)

    overall = sentiment_scores.get('overall', 0.5)
    credible_count = sentiment_scores.get('num_credible_reviews',
0)

    return (
        f"{provider_name}: {' '.join(explanations)} "
        f"(based on {credible_count} verified reviews)"
    )

# Usage Example
aggregator = SentimentAggregator()
sentiments = [
    {'punctuality': 0.9, 'quality': 0.85, 'communication': 0.8,
     'professionalism': 0.85, 'price': 0.6},
    {'punctuality': 0.7, 'quality': 0.8, 'communication': 0.75,
     'professionalism': 0.8, 'price': 0.55}
]
credibility = [0.95, 0.85]

provider_scores = aggregator.aggregate_provider_sentiments(sentiments,
credibility)
final_score = aggregator.adjust_final_ranking(0.87, provider_scores,
0.20)
explanation = aggregator.generate_explanation("Provider X",
provider_scores)

print(f"Sentiment Profile: {provider_scores}")

```

```
print(f"Final Score: {final_score:.3f}")
print(f"{explanation}")
```

Component 4 Complete Pipeline

```
class Component4ReviewCredibility:
    """Component 4: LSTM-Based Sentiment & Review Credibility"""

    def __init__(self):
        self.preprocessor = ReviewPreprocessor()
        self.extractor = AspectSentimentExtractor()
        self.detector = FakeReviewDetector()
        self.aggregator = SentimentAggregator()

    def process_provider_reviews(self, provider_id, reviews_list,
                                comp3_score):
        """Process all reviews and generate final score"""
        if not reviews_list:
            return {
                'provider_id': provider_id,
                'component4_score': comp3_score,
                'sentiment_profile': {},
                'explanation': 'No reviews available'
            }

        # Preprocess
        processed = self.preprocessor.preprocess_batch(
            [r['text'] for r in reviews_list]
        )

        # Detect fakes
        self.detector.fit_detector(reviews_list)
        fake_results = self.detector.detect_fake_reviews(reviews_list)
        credibility_scores = [r['credibility_score'] for r in
                             fake_results]

        # Extract sentiments
        tokens = [p['tokens'] for p in processed]
        sentiments_dict = self.extractor.extract_sentiments(tokens)
        sentiments_list = [sentiments_dict[i] for i in
                           range(len(reviews_list))]

        # Aggregate
        provider_sentiments =
self.aggregator.aggregate_provider_sentiments(
            sentiments_list, credibility_scores
        )

        # Final score
        final_score = self.aggregator.adjust_final_ranking(
            comp3_score, provider_sentiments, sentiment_weight=0.20
```

```

    )

    explanation = self.aggregator.generate_explanation(
        f"Provider {provider_id}", provider_sentiments
    )

    return {
        'provider_id': provider_id,
        'component4_score': final_score,
        'sentiment_profile': provider_sentiments,
        'explanation': explanation,
        'num_reviews': len(reviews_list),
        'num_credible':
provider_sentiments.get('num_credible_reviews', 0),
        'num_flagged': sum(1 for r in fake_results if
r['is_fake'])
    }

def refine_top_8_to_top_5(self, top_8, provider_reviews):
    """Final refinement to Top-5"""
    results = []

    for provider_id, comp3_score in top_8:
        reviews = provider_reviews.get(provider_id, [])
        result = self.process_provider_reviews(
            provider_id, reviews, comp3_score
        )
        results.append((provider_id, result['component4_score'],
result))

    # Sort by final score
    ranked = sorted(results, key=lambda x: x[1], reverse=True)
    top_5 = ranked[:5]

    return top_5

# Usage Example
comp4 = Component4ReviewCredibility()
top_8 = [('P001', 0.88), ('P002', 0.85)]
provider_reviews = {
    'P001': [
        {'text': 'Excellent service, on time', 'rating': 5},
        {'text': 'Good work quality', 'rating': 4}
    ],
    'P002': [
        {'text': 'AMAZING!!', 'rating': 5},
        {'text': 'Poor communication', 'rating': 2}
    ]
}

```

```

top_5 = comp4.refine_top_8_to_top_5(top_8, provider_reviews)
print("\nComponent 4 Output (Top-5 Final):")
for rank, (pid, score, result) in enumerate(top_5, 1):
    print(f"  {rank}. {pid}")
    print(f"      Final Score: {score:.4f}")
    print(f"      {result['explanation']}")

```

END-TO-END SYSTEM INTEGRATION

Complete Pipeline

```

class MLDrivenServiceMatchingPlatform:
    """Complete ML-Driven Service Provider Matching Platform"""

    def __init__(self):
        self.component1 = Component1HybridRecommendation()
        self.component2 = Component2TemporalContextual()
        self.component3 = Component3DynamicCredibility()
        self.component4 = Component4ReviewCredibility()

        # Data stores
        self.user_ratings = None
        self.provider_profiles = None
        self.provider_database = None
        self.provider_reviews = None

    def initialize_system(self, user_ratings, provider_profiles,
                        provider_database, provider_reviews):
        """Initialize with historical data"""
        self.user_ratings = user_ratings
        self.provider_profiles = provider_profiles
        self.provider_database = provider_database
        self.provider_reviews = provider_reviews

        # Train components
        self.component1.fit(user_ratings, provider_profiles)

        return self

    def get_recommendations(self, user_id, user_request, user_lat,
                          user_lon, service_type):
        """
        Get Top-5 provider recommendations for a user request

        Args:
            user_id: unique user identifier
            user_request: text description of request
            user_lat, user_lon: user geolocation
            service_type: type of service

```

```

Returns:
    top_5_recommendations: list of provider dicts with full
details
"""
print("[1/4] Component 1: Hybrid Recommendation...")
top_20 = self.component1.recommend_top_k(
    user_id,
    self.user_ratings[user_id],
    user_request,
    k=20
)
print(f"  -> Top-20 providers selected")

print("[2/4] Component 2: Temporal & Contextual Matching...")
top_10 = self.component2.refine_top_20(
    top_20,
    user_lat, user_lon,
    service_type,
    k=10
)
print(f"  -> Top-10 providers refined")

print("[3/4] Component 3: Credential Verification...")
top_8 = self.component3.refine_top_10_to_top_8(
    top_10,
    self.provider_database
)
print(f"  -> Top-8 providers verified")

print("[4/4] Component 4: Sentiment Analysis...")
top_5_results = self.component4.refine_top_8_to_top_5(
    top_8,
    self.provider_reviews
)
print(f"  -> Top-5 final recommendations with sentiment")

# Format final output
final_recommendations = []
for rank, (provider_id, final_score, comp4_data) in
enumerate(top_5_results, 1):
    final_recommendations.append({
        'rank': rank,
        'provider_id': provider_id,
        'final_score': float(final_score),
        'sentiment_profile': comp4_data['sentiment_profile'],
        'explanation': comp4_data['explanation'],
        'num_reviews': comp4_data['num_reviews']
    })

return final_recommendations

```

```

# Usage Example
platform = MLDrivenServiceMatchingPlatform()

# Initialize with data
user_ratings = np.random.rand(100, 50)
provider_profiles = [f"Provider {i}" for i in range(50)]
provider_database = {str(i): {'jobs_count': 50+i*2} for i in
range(50)}
provider_reviews = {
    str(i): [{'text': f'Review about provider {i}', 'rating': 4+i%2}]
    for i in range(50)}
}

platform.initialize_system(user_ratings, provider_profiles,
                           provider_database, provider_reviews)

# Get recommendations
recommendations = platform.get_recommendations(
    user_id=0,
    user_request="AC repair, noisy compressor",
    user_lat=6.9270,
    user_lon=79.8610,
    service_type='ac_repair'
)

print("\n" + "="*70)
print("FINAL TOP-5 RECOMMENDATIONS")
print("="*70)
for rec in recommendations:
    print(f"\n#{rec['rank']} - {rec['provider_id']}")
    print(f"    Final Score: {rec['final_score']:.4f}")
    print(f"    Reviews Analyzed: {rec['num_reviews']}")
    print(f"    {rec['explanation']}")

```

DATABASE SCHEMA

Core Tables

users

```

CREATE TABLE users (
    user_id VARCHAR(50) PRIMARY KEY,
    name VARCHAR(255),
    email VARCHAR(255),
    phone VARCHAR(20),
    latitude FLOAT,
    longitude FLOAT,
    created_at TIMESTAMP,

```

```
        updated_at TIMESTAMP
    );
```

providers

```
CREATE TABLE providers (
    provider_id VARCHAR(50) PRIMARY KEY,
    name VARCHAR(255),
    email VARCHAR(255),
    phone VARCHAR(20),
    service_type VARCHAR(100),
    description TEXT,
    latitude FLOAT,
    longitude FLOAT,
    years_experience INT,
    is_online BOOLEAN,
    created_at TIMESTAMP
);
```

reviews

```
CREATE TABLE reviews (
    review_id VARCHAR(50) PRIMARY KEY,
    job_id VARCHAR(50),
    provider_id VARCHAR(50),
    user_id VARCHAR(50),
    rating INT,
    text TEXT,
    created_at TIMESTAMP,
    FOREIGN KEY (provider_id) REFERENCES providers(provider_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

bookings

```
CREATE TABLE bookings (
    job_id VARCHAR(50) PRIMARY KEY,
    user_id VARCHAR(50),
    provider_id VARCHAR(50),
    service_type VARCHAR(100),
    request_text TEXT,
    status VARCHAR(50), -- PENDING, ACCEPTED, COMPLETED
    created_at TIMESTAMP,
    completed_at TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (provider_id) REFERENCES providers(provider_id)
);
```


SYSTEM EVALUATION METRICS

Component-Level Metrics

Component	Metric	Target	Formula
Component 1	Accuracy@5	85%	$\frac{\text{True Positives}}{\text{Top-5}} \times 10$
	Precision@20	90%	$\frac{\text{High-Rated Providers}}{\text{Top-20}}$
	NDCG@5	0.85	Ranking quality metric
Component 2	Geoloc. Accuracy	90%	Distance prediction error
	Availability Match	88%	Providers available when predicted
Component 3	Credential Detection	88%	Fake credential detection
	Fraud Detection	85%	Fraudulent provider detection
Component 4	Sentiment F1	0.85	Aspect extraction accuracy
	Fake Review Detection	0.90	Precision in fake review identification
End-to-End	User Satisfaction	82%	User satisfaction survey

Evaluation Code

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

class PlatformEvaluator:
    """Evaluate platform performance"""

    @staticmethod
    def evaluate_component1(predictions, ground_truth):
        """Evaluate Component 1 recommendations"""
        # Top-5 accuracy
        top5_acc = accuracy_score(ground_truth[:5], predictions[:5])

        # Precision@20
        highRated = sum(1 for p in predictions[:20] if p >= 4.0)
```

```

precision20 = highRated / 20

return {'top5_accuracy': top5_acc, 'precision20': precision20}

@staticmethod
def evaluate_component4_sentiment(predicted_sentiments,
                                  ground_truth_sentiments):
    """Evaluate sentiment extraction"""
    f1 = f1_score(ground_truth_sentiments, predicted_sentiments,
                  average='weighted')
    return {'sentiment_f1': f1}

# Usage
evaluator = PlatformEvaluator()
comp1_metrics = evaluator.evaluate_component1(predictions,
ground_truth)
print(f"Component 1 Metrics: {comp1_metrics}")

```

CONCLUSION

This comprehensive guide provides complete technical implementation details for a 4-component ML-driven service provider matching platform. The system progressively filters and ranks providers through intelligent stages:

1. **Component 1** generates skill-based Top-20 recommendations
2. **Component 2** refines to availability-aware Top-10
3. **Component 3** verifies credibility for Top-8
4. **Component 4** analyzes sentiment for final Top-5

Key innovations include:

- Hybrid multi-algorithm fusion (CF + CBF + BERT)
- Real-time contextual matching (availability, location, weather)
- Dynamic credibility assessment with fraud detection
- LSTM-based aspect-level sentiment analysis
- Fake review detection using anomaly detection

Expected performance targets:

- Top-5 Accuracy: 85%
- User Satisfaction: 82%
- Response Time: <10 seconds

- System Reliability: >99.5%